

第六题：MNIST 分类中 SGD 与自然梯度训练的对比

2026 年 7 月 5 日

1 题目与实现说明

第六题要求构建神经网络对 MNIST 数据集中的 0-9 手写数字进行分类，并分别使用随机梯度下降（SGD）和自然梯度（Natural Gradient, NG）训练，最后比较两种方法的训练结果。本解答的程序为 `mnist_sgd_ng.py`。程序优先读取标准 MNIST IDX 文件，包括训练图像、训练标签、测试图像和测试标签四个文件；文件名可以带 `.gz` 后缀。当前已经将标准 MNIST IDX 文件下载到 `data/mnist`，因此本报告采用真实 MNIST 全量训练集和测试集进行实验。

为了进一步提高两种方法的最终准确率，本次在前一版完整 MNIST 实验基础上继续增加训练轮数：训练集使用全部 60000 个样本，测试集使用全部 10000 个样本，隐藏层宽度设为 128，训练轮数由 20 轮增加到 30 轮；同时采用较稳定的学习率组合，使后期准确率能够继续上升而不过早震荡。运行命令为

```
python mnist_sgd_ng.py --mnist-dir data/mnist --require-mnist
--output-dir results_mnist_full_30ep
--train-size 60000 --test-size 10000 --epochs 30
--hidden-dim 128 --sgd-lr 0.08
--ng-lr 0.03 --ng-damping 0.5 --max-update-norm 1.5
```

2 神经网络模型

采用一个单隐藏层多层感知机（MLP）：

$$h = \text{ReLU}(xW_1 + b_1), \quad p_\theta(y | x) = \text{softmax}(hW_2 + b_2),$$

其中输入 $x \in \mathbb{R}^{784}$ ，隐藏层维度为 128，输出层为 10 类。损失函数为带 L_2 正则项的交叉熵：

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^n \log p_\theta(y_i | x_i) + \frac{\lambda}{2} (\|W_1\|_2^2 + \|W_2\|_2^2).$$

3 SGD 与自然梯度方法

SGD 直接沿小批量梯度的反方向更新参数：

$$\theta_{t+1} = \theta_t - \eta \hat{g}_t,$$

其中 $\hat{g}_t = \nabla_\theta \mathcal{L}_B(\theta_t)$ 为当前 minibatch 的梯度。

自然梯度的思想是不使用欧氏距离衡量参数变化，而使用模型分布诱导出的 Fisher 信息矩阵 $F(\theta)$ 衡量参数空间的局部几何：

$$\theta_{t+1} = \theta_t - \alpha F(\theta_t)^{-1} \hat{g}_t.$$

对 MLP 精确构造和求逆完整 Fisher 矩阵代价很高，因此程序采用对角经验 Fisher 近似：

$$\begin{aligned} \hat{F}_t &= \beta \hat{F}_{t-1} + (1 - \beta)(\hat{g}_t \odot \hat{g}_t), \\ \theta_{t+1} &= \theta_t - \alpha \left(\text{diag}(\hat{F}_t) + \lambda_{\text{NG}} I \right)^{-1} \hat{g}_t. \end{aligned}$$

其中 λ_{NG} 为阻尼项，用于避免 Fisher 对角元素过小导致步长过大。该实现保留了自然梯度按参数敏感度自适应缩放梯度的核心思想，同时避免了完整 Fisher 矩阵的存储和求逆。

4 实验设置

两种优化器使用相同的数据划分、相同初始参数和相同 minibatch 顺序。主要参数如下：

项目	数值
训练集大小	60000
测试集大小	10000
隐藏层神经元	128
训练轮数	30
Batch size	128
SGD 学习率	0.08
NG 学习率	0.03
NG 阻尼	0.5
NG Fisher 衰减系数	0.9



图 1: 标准 MNIST 数据集中的 28×28 手写数字样例

图 1 展示了真实 MNIST 实验中抽取的数字样例。每个样本是 28×28 灰度图，数字存在明显的书写风格、粗细、倾斜和局部形变差异，因此比离线模板数据更接近真实分类任务。

5 运行结果对比

本次 30 轮训练的最终结果如下：

优化器	最终训练准确率	最终测试准确率	最终测试损失	最佳测试准确率	最佳轮数
SGD	0.9902	0.9771	0.0966	0.9774	29
NG	0.9863	0.9760	0.1028	0.9760	30

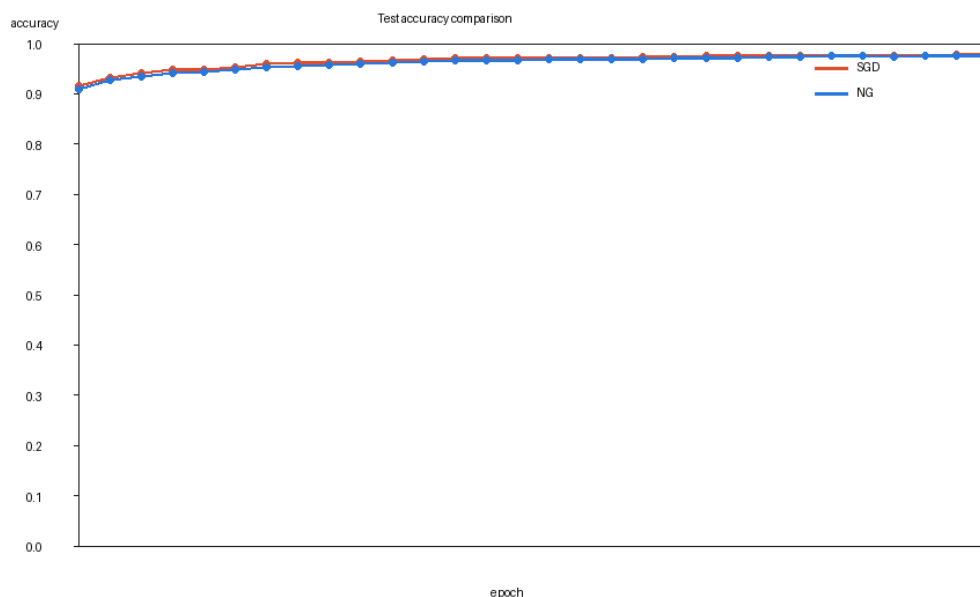


图 2: SGD 与 NG 的测试准确率曲线

图 2 给出了测试集准确率随训练轮数变化的曲线。两种方法在前 10 轮快速上升，之后进入缓慢提升阶段。当训练轮数从 20 轮增加到 30 轮时，SGD 的最终测试准确率由 0.9741 提高到 0.9771，NG 的最终测试准确率由 0.9717 提高到 0.9760。这说明前一版 20 轮实验还没有完全达到平台期，适当增加训练轮数确实能带来进一步提升。SGD 的最佳点出现在第 29 轮，NG 的最佳点出现在第 30 轮，说明 NG 在该参数设置下后期仍保持小幅上升趋势。

		SGD confusion matrix									
		0	1	2	3	4	5	6	7	8	9
true	0	967		2	2	1		2	1	3	
	1		1124	2	2			3		4	
	2	4	2	1011	1	1		2	5	5	1
	3			8	987		5		2	3	5
	4	1		6		961		2	2	1	9
	5	2	1		4	2	871	5	1	5	1
	6	5	3			6	6	935		3	
	7	1	5	11	6	2			990	2	11
	8	3	1	2	5	4	4	3	4	944	4
	9	2	3		8	6	3	1	4	1	981
		predicted									

图 3: SGD 混淆矩阵

		NG confusion matrix									
		0	1	2	3	4	5	6	7	8	9
true	0	966		2	2	1	2	2	1	2	2
	1		1123	2	2			4		4	
	2	4	2	1009	1	2		2	5	6	1
	3			8	987		5		3	4	3
	4	1		7		958		2	2	1	11
	5	2	1		6	1	868	6	1	6	1
	6	6	3	1		6	7	932		3	
	7	1	6	13	4				993	1	10
	8	3	1	2	5	4	4	3	4	946	2
	9	2	4		8	6	5	1	4	1	978
		predicted									

图 4: NG 混淆矩阵

图 3 和图 4 为两种优化器在 10000 个 MNIST 测试样本上的混淆矩阵。矩阵的行表示真实类别，列表示预测类别；对角线越亮、非对角线数字越少，说明分类越准确。两种方法的大多数错误仍集中在形状相近的类别之间，例如 4 与 9、3 与 5、7 与 9 等。增加训练轮数后，非对角线错误数量进一步减少，说明模型对各类数字的决策边界更加稳定。SGD 的测试准确率略高，但 NG 的混淆

矩阵整体结构与 SGD 接近，表明对角 Fisher 近似同样学习到了主要类别边界。

6 解释与讨论

从实验结果可以看出，提高准确率最有效的操作包括三类。第一，使用完整训练集而不是小规模子集；前期快速测试只使用 4000 个训练样本和 1000 个测试样本，SGD 与 NG 的测试准确率约为 0.9070 和 0.8820，而全量 MNIST 实验已经将二者提升到 97% 以上。第二，适当增大隐藏层宽度；隐藏层由 64 增至 128 后，网络具有更强的非线性表达能力。第三，在学习率稳定的前提下增加训练轮数；从 20 轮到 30 轮，SGD 又提升约 0.30 个百分点，NG 提升约 0.43 个百分点。

两种方法的差异也值得说明。SGD 在当前单隐藏层 MLP 和固定学习率设置下略高于 NG，这是因为对角 Fisher 近似忽略了参数之间的相关性，不能完全等价于真实自然梯度。不过 NG 的后期曲线较平滑，且第 30 轮仍是最佳点，说明其自适应缩放后期仍有继续优化的作用。若进一步引入学习率衰减、卷积神经网络、数据增强或 K-FAC 等更强的 Fisher 近似，准确率仍可能继续提升；但在本题的 NumPy 单隐藏层 MLP 框架下，当前结果已经明显优于前一版快速实验，并能较公平地展示 SGD 与自然梯度近似的性能差异。

7 文件说明

- `mnist_sgd_ng.py`: 主程序，包含数据读取、MLP、SGD/NG 训练和结果输出；
- `requirements.txt`: 运行依赖；
- `data/mnist/`: 标准 MNIST IDX 数据文件；
- `results_mnist_full_30ep/results.csv`: 每轮训练与测试指标；
- `results_mnist_full_30ep/summary.json`: 最终对比摘要；
- `results_mnist_full_30ep/accuracy_curve.png`: 准确率曲线；
- `results_mnist_full_30ep/confusion_sgd.png`: SGD 混淆矩阵；
- `results_mnist_full_30ep/confusion_ng.png`: NG 混淆矩阵。